

Reliable and Trustworthy Artificial Intelligence

Lecture 4: DeepPoly Convex Relaxation

Martin Vechev
ETH Zurich

Fall 2026

Scaling Deterministic Certification: VNN-COMP

Note: Before using more expensive methods, we always try cheaper methods. We try Box, Box(intermediate) + DeepPoly (final bounds), full DeepPoly, DeepPoly with slope optimization, BaB with DeepPoly in this order.

Benchmarks 2025 - Regular Track https://github.com/VNN-COMP/vnncomp2025_benchmarks

Name	Network Type	# Params	Input Dim	# Regions
safeNLP	FC + ReLU	4k	30	1
nn4sys	Conv, FC, Residual + ReLU, Sigmoid	33k - 37M	1-308	1 - 11k
cora	FC + ReLU	0.5M - 1.1M	784-3072	9
LinearizeNN	FC, Residual + ReLU	203k	4	2
Dist Shift	FC + ReLU, Sigmoid	342k-855k	792	1
cifar100	FC, Conv, Residual + ReLU, BatchNorm	2.5M - 3.8M	3072	99
tinyimagenet	FC, Conv, Residual + ReLU, BatchNorm	3.6M	9408	199
Acas XU	FC + ReLU	13k	5	1-4
cGAN	FC, Conv, ConvTranspose, Residual + ReLU, BatchNorm, AvgPool	500k-68M	5	2
Collins Rul CNN	Conv + ReLU, Dropout	60k - 262k	400-800	2
Metaroom	Conv, FC + ReLU	466k-7.4M	5376	19
TLL Verify Bench	FC + ReLU	17k - 67M	2	1

Detailed report (2025):

<https://docs.google.com/presentation/d/1ep-hGGotgWQF6SA0JlpQ6nFqs2lXoyuLMM-bORzNvrQ/>

Web Site (2026):

<https://vnn-comp.github.io/>

The problem with Box

Box is **simple and efficient**, but the problem is that it **loses too much precision**, in **both**, the ReLU abstract transformer but also in the affine abstract transformer.

The Polyhedral convex relaxation we study now is such that the **affine transformer will not lose precision** (that is, it will be precise); however, its ReLU transformer will again lose some precision.

DeepPoly convex relaxation: the shape

[Singh et. al, POPL'19]

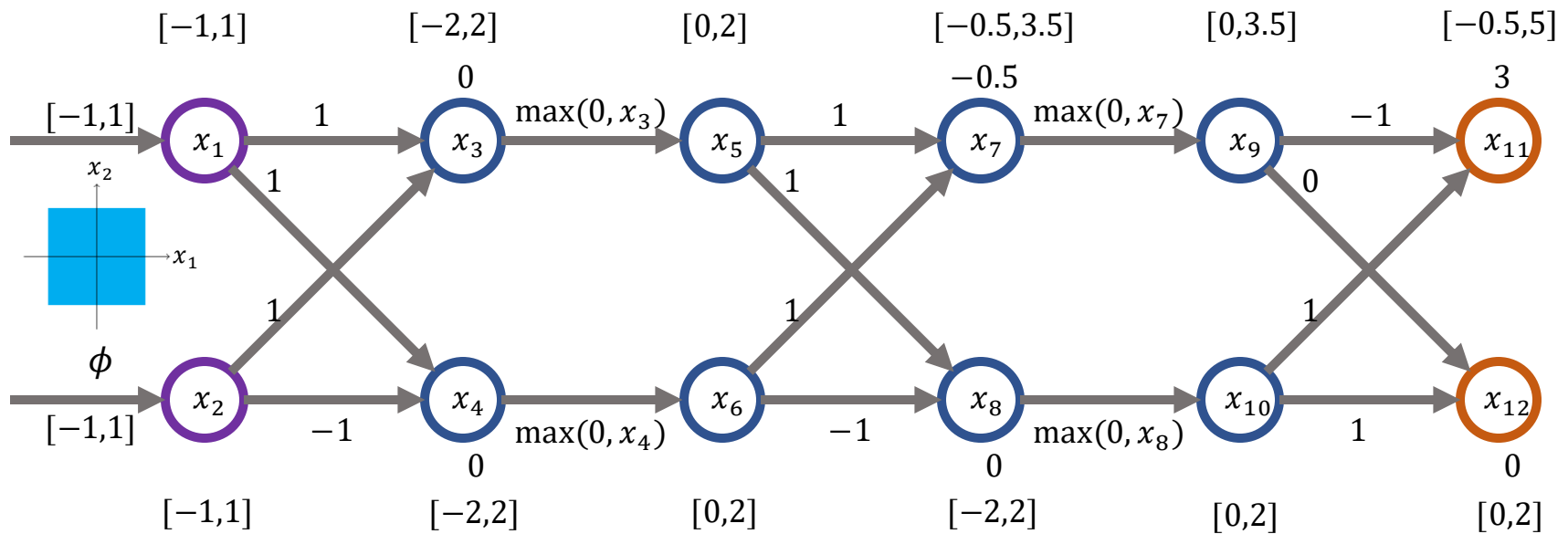
Shape:

for each x_i , we keep:

- An **interval constraint**: lower bound $l_i \leq x_i$ and upper bound $x_i \leq u_i$
- Two **relational constraints**: $a_i^{\leq} \leq x_i$ and $x_i \leq a_i^{\geq}$
where the expressions $a_i^{\leq}, a_i^{\geq}$ are of the form $\sum_j w_j \cdot x_j + v$

- less precise than Polyhedra, restriction needed to ensure scalability
- captures affine transformations precisely
- custom transformers for ReLU, sigmoid, tanh, and maxpool activations

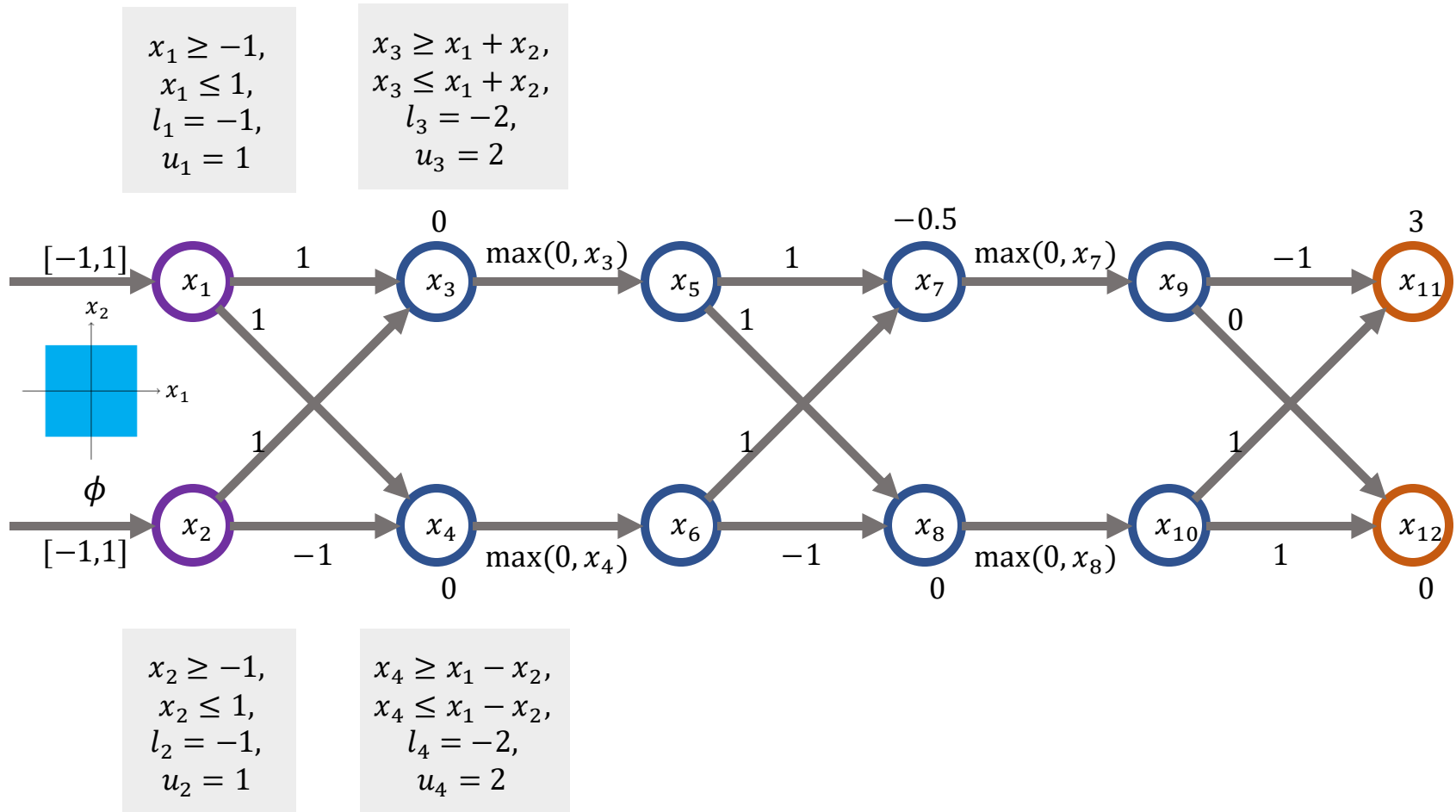
Box relaxation (scalable but imprecise)



ψ : we want to prove that $x_{11} > x_{12}$ for all values of x_1, x_2 in the input set

Certification with Box fails as it cannot capture relational information

DeepPoly relaxation



ReLU activation: $x_j := \max(0, x_i)$

Single-neuron transformer for $x_j := \max(0, x_i)$ that uses l_i, u_i

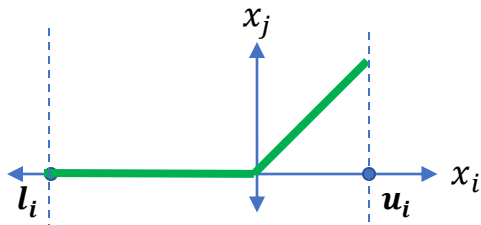
ReLU activation: $x_j := \max(0, x_i)$

Single-neuron transformer for $x_j := \max(0, x_i)$ that uses l_i, u_i

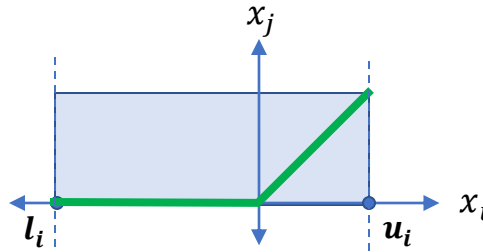
- *if $u_i \leq 0$: $a_j^{\leq} = a_j^{\geq} = 0, l_j = u_j = 0$* (strictly negative)
- *if $l_i \geq 0$: $a_j^{\leq} = a_j^{\geq} = x_i, l_j = l_i, u_j = u_i$* (strictly positive)
- *if $l_i < 0$ and $u_i > 0$* (crossing ReLU)

Lets discuss the crossing ReLU activation next

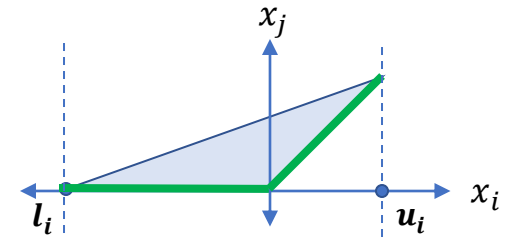
ReLU activation: $x_j := \max(0, x_i)$



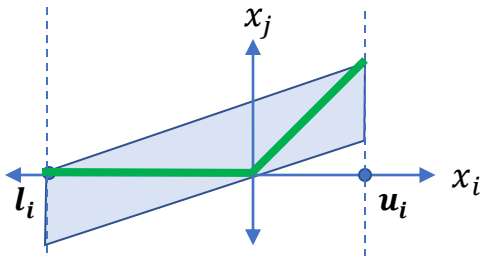
Exact [Katz et al., CAV'17]



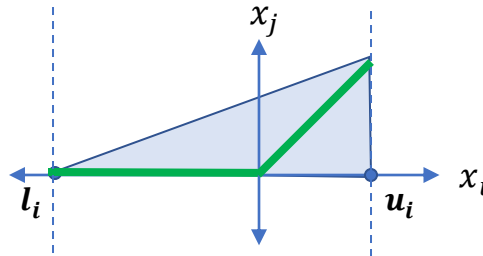
Box [Gehr et al. S&P'18]



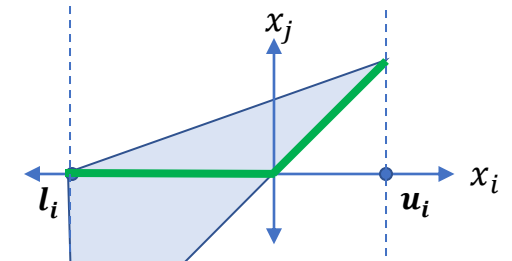
Triangle [Ehlers et al. ATVA'17]



Zonotope:
Wong et al. [ICML'18]
FastLin [ICML'18]
DeepZ [NeurIPS'18]



CROWN [NeurIPS'18]
DeepPoly [POPL'19]



CROWN [NeurIPS'18]
DeepPoly [POPL'19]

DeepPoly ReLU Relaxation I

This line is:

$$\begin{aligned} y &= \lambda * x - \lambda * l_x \\ &= \lambda * (x - l_x) \end{aligned}$$

The slope is:

$$\lambda = \frac{u_x}{u_x - l_x}$$

So the blue area approximating the green line is captured by 2 inequality constraints below:

$$\begin{aligned} y &\leq \lambda * (x - l_x) \\ y &\geq 0 \end{aligned}$$

as well as the upper bound constraint kept as part of the previous layer:

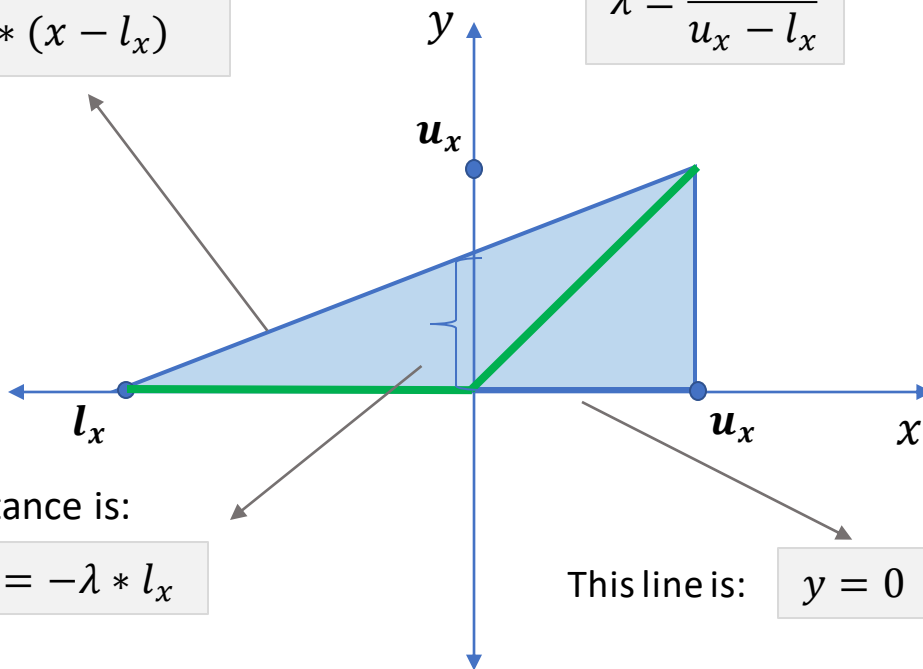
$$x \leq u_x$$

distance is:

$$d = -\lambda * l_x$$

This line is:

$$y = 0$$



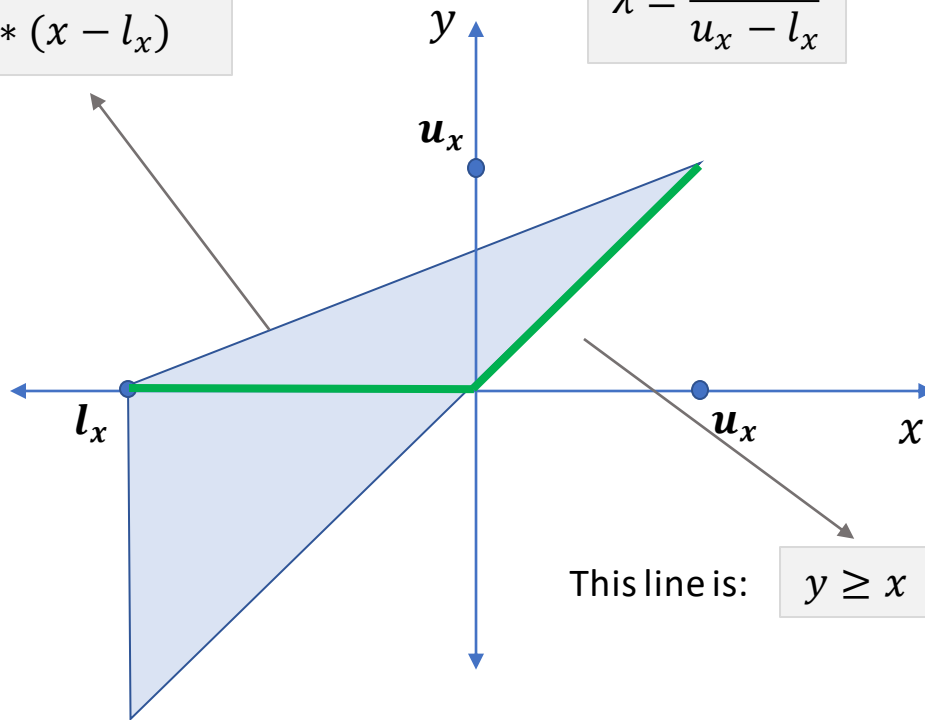
DeepPoly ReLU Relaxation II

This line is (the same as before):

$$\begin{aligned}y &= \lambda * x - \lambda * l_x \\ &= \lambda * (x - l_x)\end{aligned}$$

The slope is:

$$\lambda = \frac{u_x}{u_x - l_x}$$



So the blue area approximating the green line is captured by 2 inequality constraints below:

$$\begin{aligned}y &\leq \lambda * (x - l_x) \\ y &\geq x\end{aligned}$$

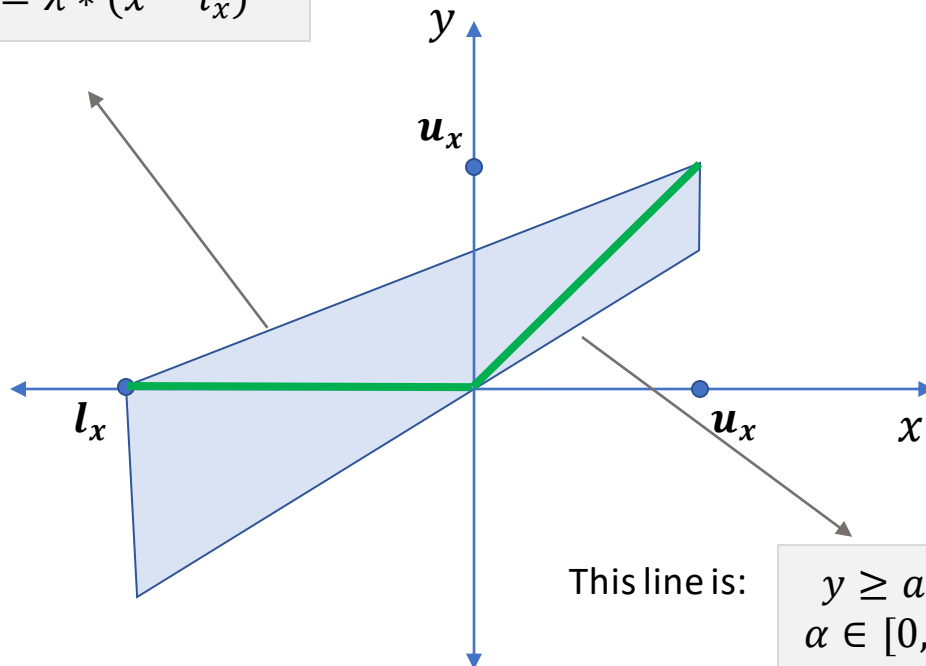
as well as the upper bound constraint kept as part of the previous layer:

$$x \leq u_x$$

DeepPoly ReLU Relaxation α

This line stays the same:

$$\begin{aligned} y &= \lambda * x - \lambda * l_x \\ &= \lambda * (x - l_x) \end{aligned}$$



This line is:

$$\begin{aligned} y &\geq ax \\ \alpha &\in [0,1] \end{aligned}$$

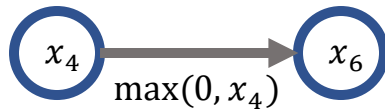
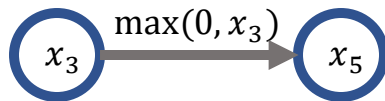
We can **vary the lower bound** constraint $y \geq \alpha x$, $\alpha \in [0,1]$ to get different non-comparable relaxations.

We can **even learn α** to get one more likely to verify the property

(note: useful for the project)

Let us apply ReLU relaxation I

$$\begin{aligned}x_3 &\geq x_1 + x_2, \\x_3 &\leq x_1 + x_2, \\l_3 &= -2, \\u_3 &= 2\end{aligned}$$



$$\begin{aligned}x_4 &\geq x_1 - x_2, \\x_4 &\leq x_1 - x_2, \\l_4 &= -2, \\u_4 &= 2\end{aligned}$$

$$\begin{aligned}x_5 &\leq \lambda * (x_3 - l_{x_3}) \\x_5 &\geq 0\end{aligned}$$

becomes

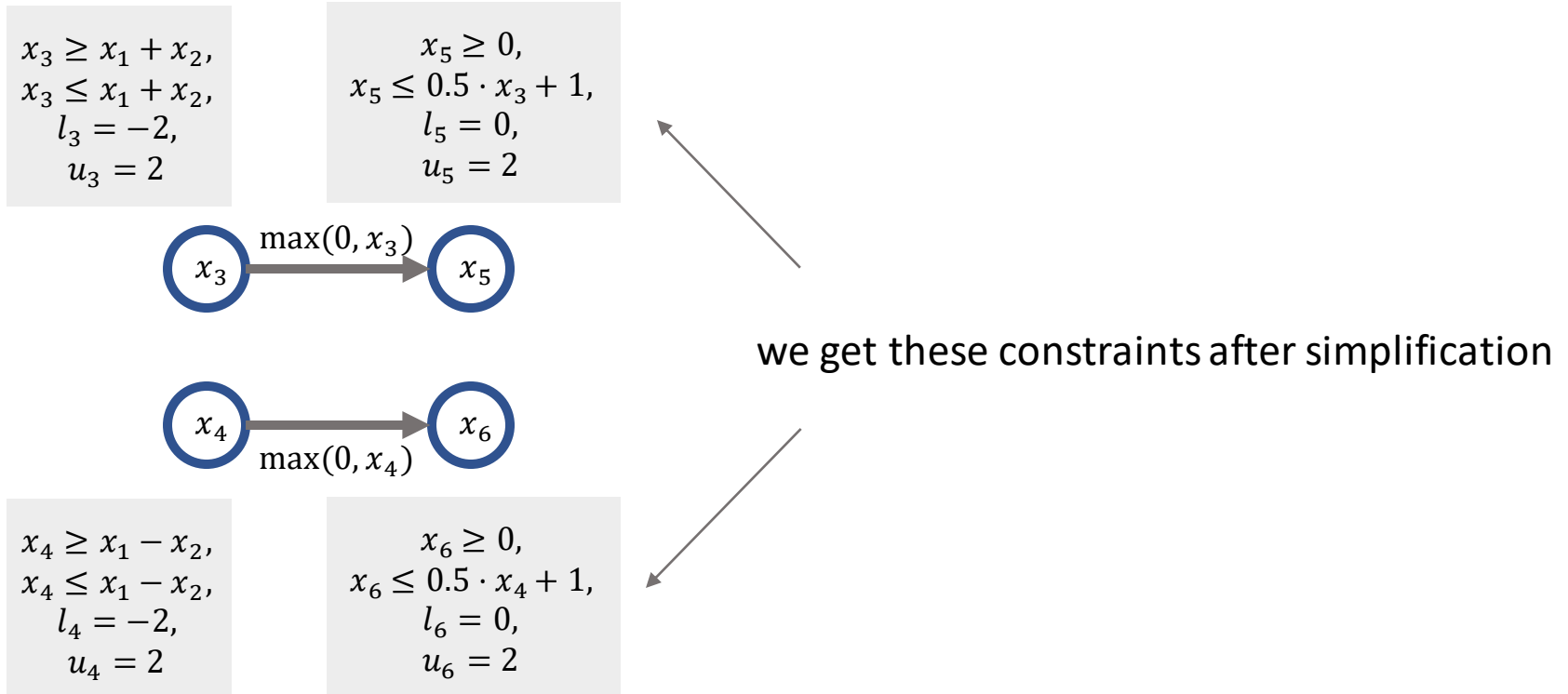
$$\begin{aligned}x_5 &\leq 0.5 * (x_3 + 2) \\x_5 &\geq 0\end{aligned}$$

$$\begin{aligned}x_6 &\leq \lambda * (x_4 - l_{x_4}) \\x_6 &\geq 0\end{aligned}$$

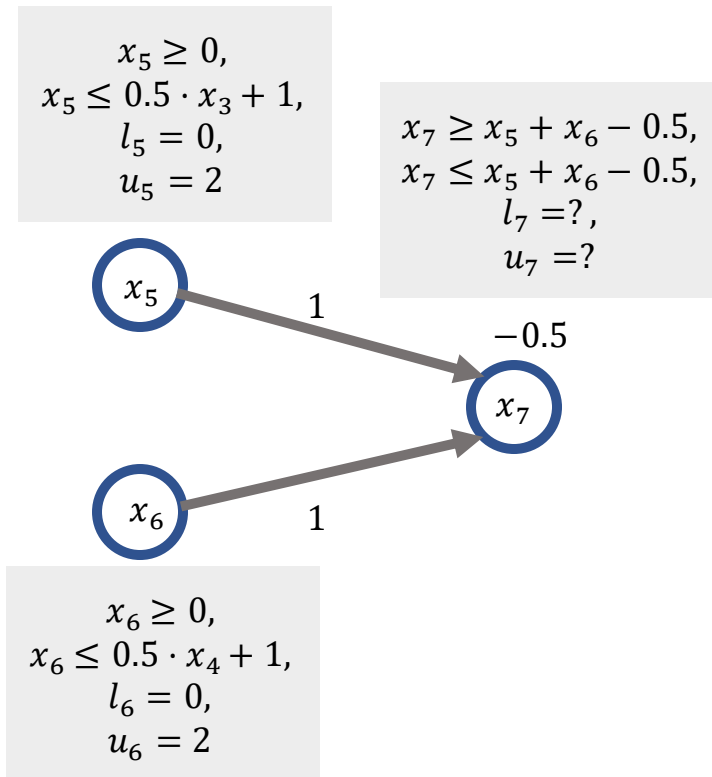
becomes

$$\begin{aligned}x_6 &\leq 0.5 * (x_4 + 2) \\x_6 &\geq 0\end{aligned}$$

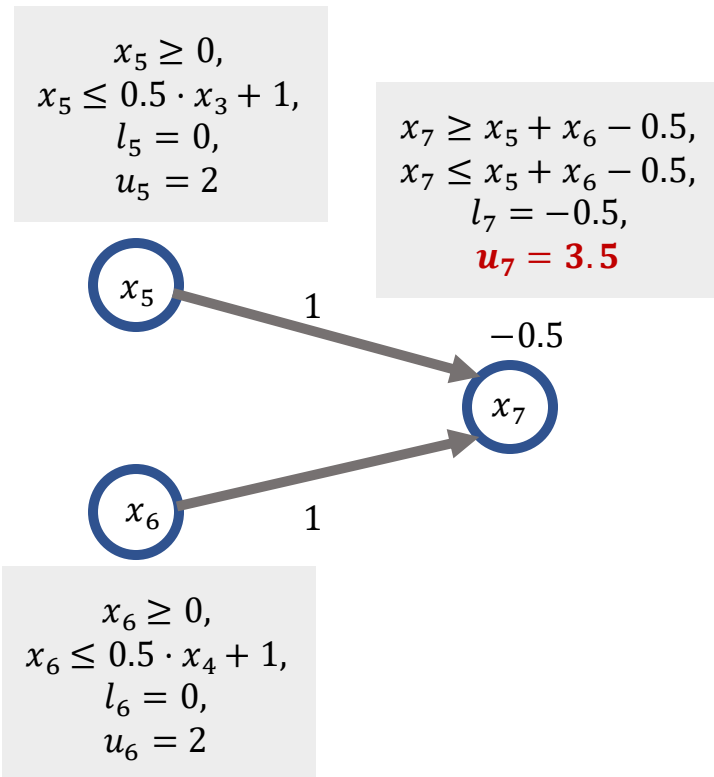
Let us apply ReLU relaxation I



Affine transformation after ReLU



Affine transformation after ReLU



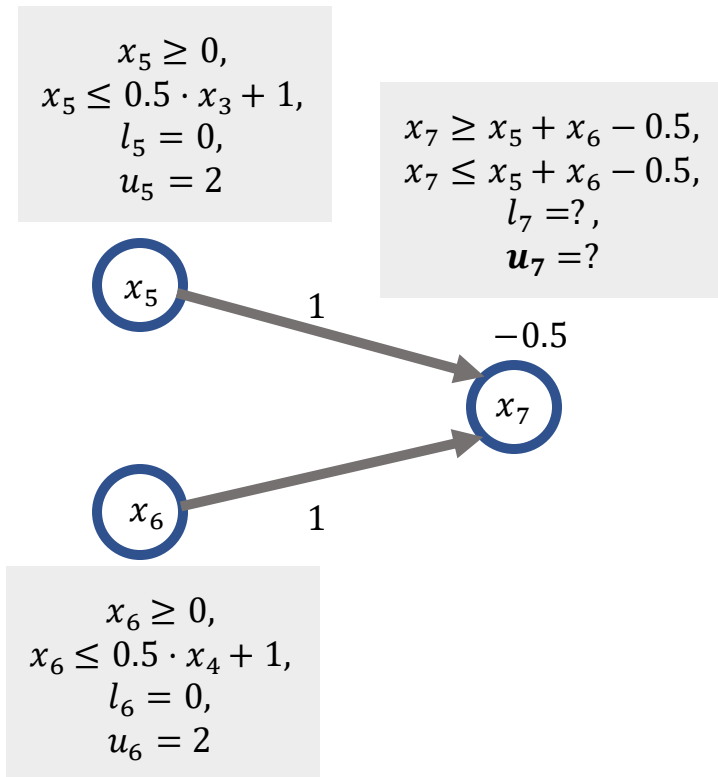
Computing upper bound for neuron x_7 by using upper bounds for u_5 and u_6 :

$$\begin{aligned} x_7 &\leq x_5 + x_6 - 0.5 \\ x_7 &\leq 2 + 2 - 0.5 \\ x_7 &\leq 3.5 \end{aligned}$$

Imprecise upper bound u_7 by substituting u_5, u_6 for x_5 and x_6

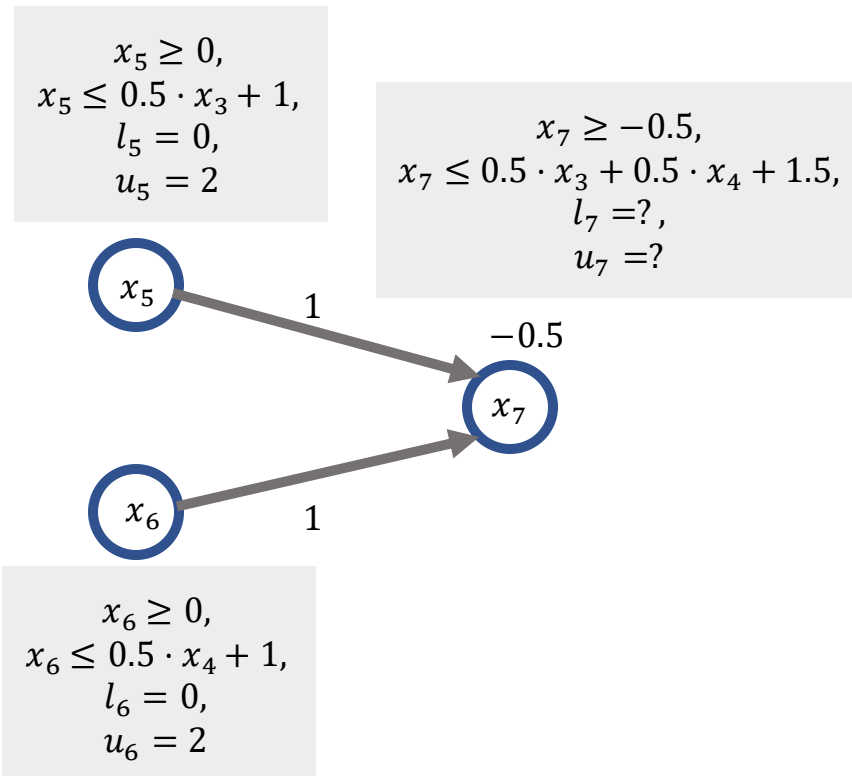
We are trying to compute **sound and precise/tight lower and upper bounds**. To do so, instead of only using the concrete l and u bounds from the previous layer, we can use the relational constraints accumulated in the prior layers. We accomplish this via **backsubstitution**.

Backsubstitution



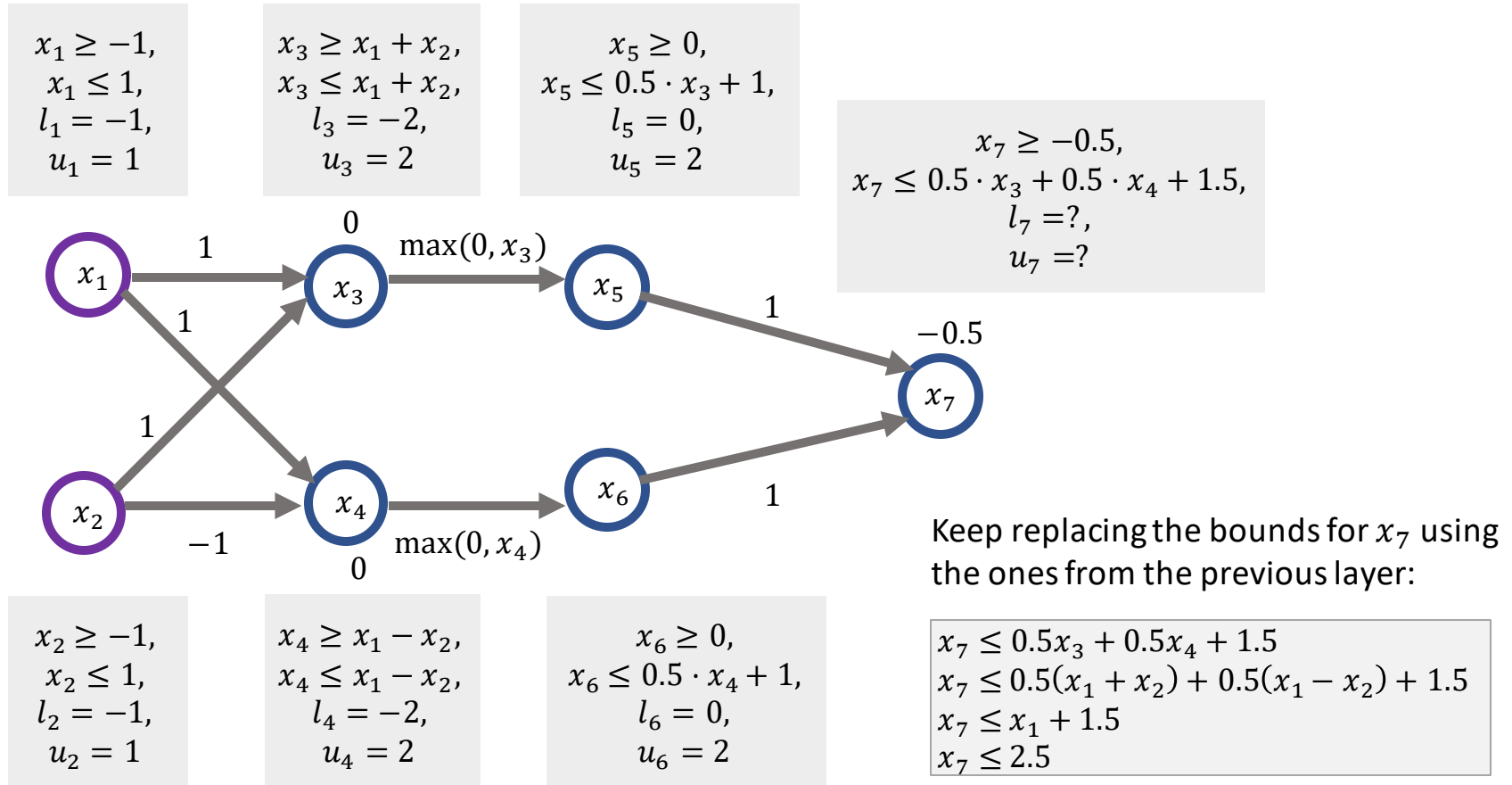
Obtain the lower and upper bounds for x_7 using the ones from the previous layer

Backsubstitution

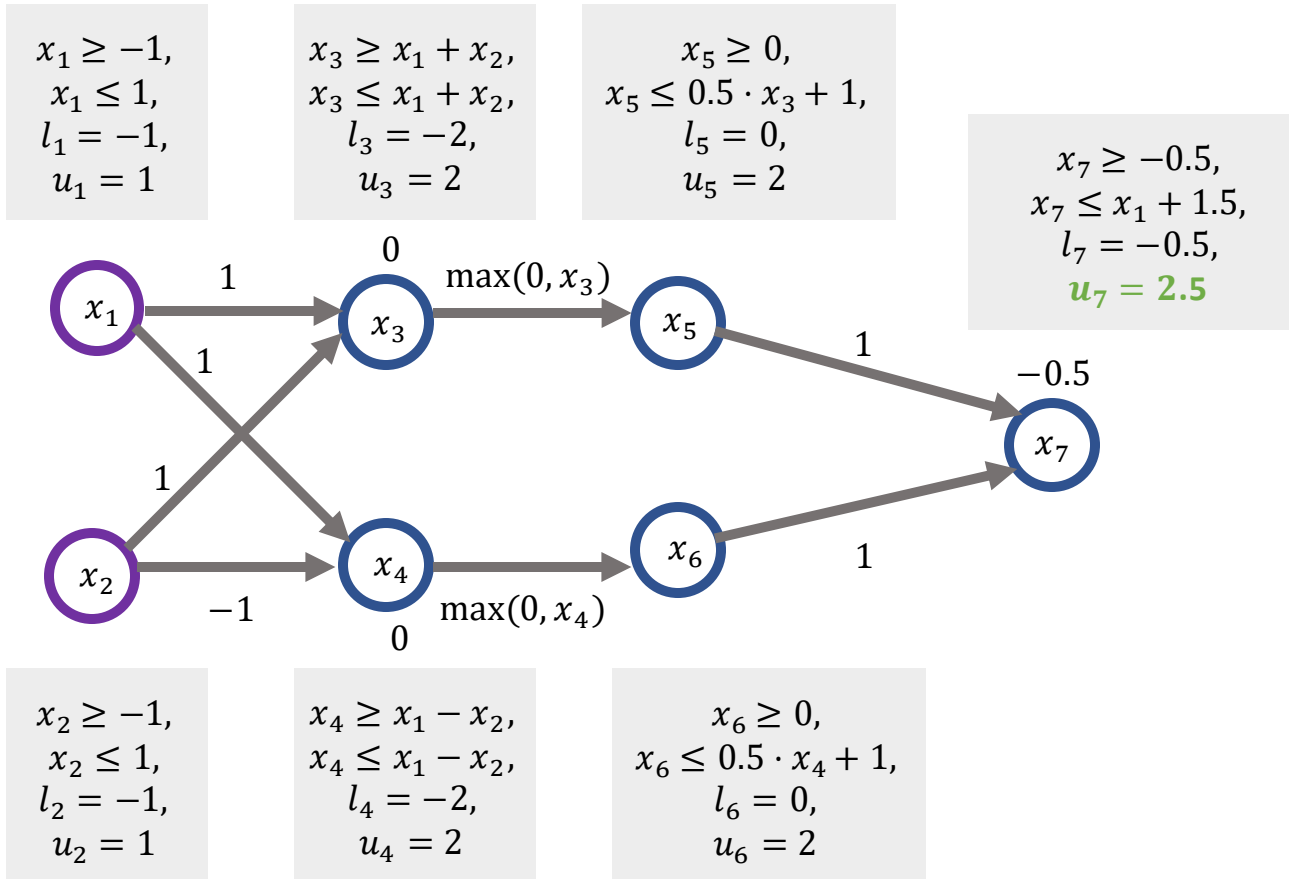


Obtain the lower and upper bounds for x_7 using the ones from the previous layer

Backsubstitution



Backsubstitution



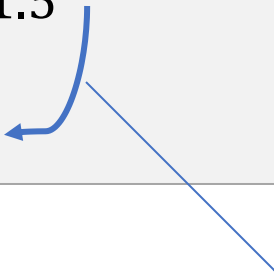
Dealing with arbitrary input regions

Reminder: in our backsubstitution, we obtained the following steps:

$$x_7 \leq 0.5x_3 + 0.5x_4 + 1.5$$

$$x_7 \leq 0.5(x_1 + x_2) + 0.5(x_1 - x_2) + 1.5$$

$$x_7 \leq x_1 + 1.5$$

$$x_7 \leq 2.5$$


But this step was possible in general as we were dealing with a very specific input region (where each neuron lies in an interval, in this case $[-1,1]$).

How can we deal with any norm? [*Holder inequality]

$$\begin{aligned} x_1 &\geq -1, \\ x_1 &\leq 1, \\ l_1 &= -1, \\ u_1 &= 1 \end{aligned}$$

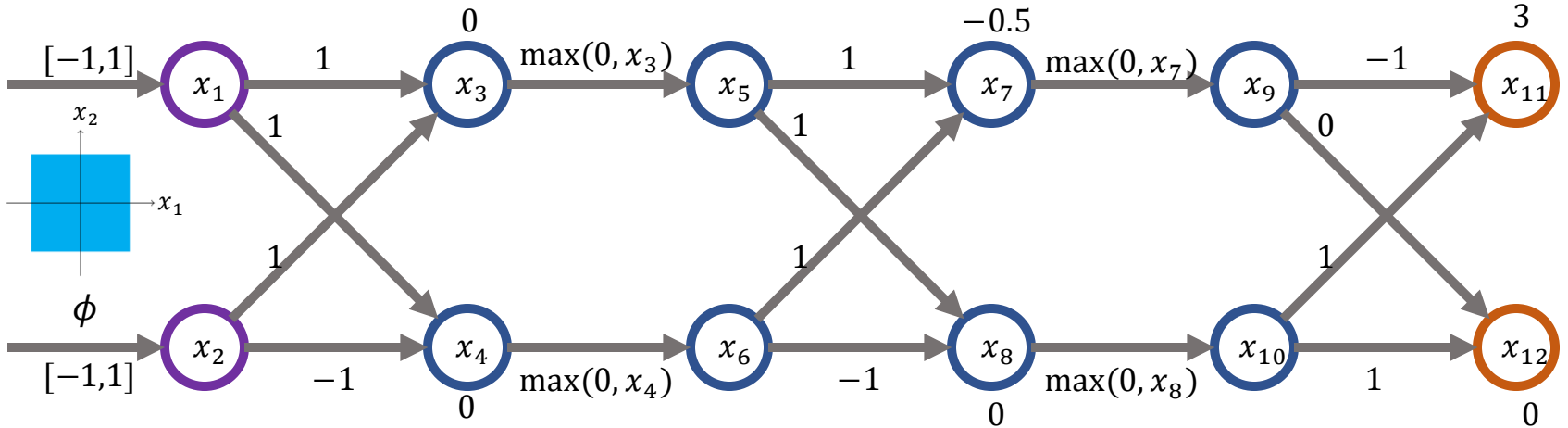
$$\begin{aligned} x_3 &\geq x_1 + x_2, \\ x_3 &\leq x_1 + x_2, \\ l_3 &= -2, \\ u_3 &= 2 \end{aligned}$$

$$\begin{aligned} x_5 &\geq 0, \\ x_5 &\leq 0.5 \cdot x_3 + 1, \\ l_5 &= 0, \\ u_5 &= 2 \end{aligned}$$

$$\begin{aligned} x_7 &\geq x_5 + x_6 - 0.5, \\ x_7 &\leq x_5 + x_6 - 0.5, \\ l_7 &= -0.5, \\ u_7 &= 2.5 \end{aligned}$$

$$\begin{aligned} x_9 &\geq 0, \\ x_9 &\leq \frac{5}{6} \cdot x_7 + \frac{5}{12}, \\ l_9 &= 0, \\ u_9 &= 2.5 \end{aligned}$$

$$\begin{aligned} x_{11} &\geq -x_9 + x_{10} + 3, \\ x_{11} &\leq -x_9 + x_{10} + 3, \\ l_{11} &= 0.5, \\ u_{11} &= 5 \end{aligned}$$



$$\begin{aligned} x_2 &\geq -1, \\ x_2 &\leq 1, \\ l_2 &= -1, \\ u_2 &= 1 \end{aligned}$$

$$\begin{aligned} x_4 &\geq x_1 - x_2, \\ x_4 &\leq x_1 - x_2, \\ l_4 &= -2, \\ u_4 &= 2 \end{aligned}$$

$$\begin{aligned} x_6 &\geq 0, \\ x_6 &\leq 0.5 \cdot x_4 + 1, \\ l_6 &= 0, \\ u_6 &= 2 \end{aligned}$$

$$\begin{aligned} x_8 &\geq x_5 - x_6, \\ x_8 &\leq x_5 - x_6, \\ l_8 &= -2, \\ u_8 &= 2 \end{aligned}$$

$$\begin{aligned} x_{10} &\geq 0, \\ x_{10} &\leq 0.5 \cdot x_8 + 1, \\ l_{10} &= 0, \\ u_{10} &= 2 \end{aligned}$$

$$\begin{aligned} x_{12} &\geq x_{10}, \\ x_{12} &\leq x_{10}, \\ l_{12} &= 0, \\ u_{12} &= 2 \end{aligned}$$

Proving the robustness property

Goal: Prove $x_{11} - x_{12} > 0$ for all inputs in $[-1,1] \times [-1,1]$

$$\begin{aligned}x_{11} &\geq -x_9 + x_{10} + 3, \\x_{11} &\leq -x_9 + x_{10} + 3, \\l_{11} &= 0.5, \\u_{11} &= 5\end{aligned}$$

$$\begin{aligned}x_{12} &\geq x_{10}, \\x_{12} &\leq x_{10}, \\l_{12} &= 0, \\u_{12} &= 2\end{aligned}$$

Proving the robustness property

Goal: Prove $x_{11} - x_{12} > 0$ for all inputs in $[-1,1] \times [-1,1]$

$$\begin{aligned}x_{11} &\geq -x_9 + x_{10} + 3, \\x_{11} &\leq -x_9 + x_{10} + 3, \\l_{11} &= 0.5, \\u_{11} &= 5\end{aligned}$$

$$\begin{aligned}x_{12} &\geq x_{10}, \\x_{12} &\leq x_{10}, \\l_{12} &= 0, \\u_{12} &= 2\end{aligned}$$

Computing lower bound for $x_{11} - x_{12}$ using l_{11}, u_{12} gives -1.5 which is an imprecise result

Proving the robustness property

Goal: Prove $x_{11} - x_{12} > 0$ for all inputs in $[-1,1] \times [-1,1]$

$$\begin{aligned}x_{11} &\geq -x_9 + x_{10} + 3, \\x_{11} &\leq -x_9 + x_{10} + 3, \\l_{11} &= 0.5, \\u_{11} &= 5\end{aligned}$$

$$\begin{aligned}x_{12} &\geq x_{10}, \\x_{12} &\leq x_{10}, \\l_{12} &= 0, \\u_{12} &= 2\end{aligned}$$

Computing lower bound for $x_{11} - x_{12}$ using l_{11}, u_{12} gives -1.5 which is an imprecise result

We now backsubstitute $x_{11} - x_{12}$, and get 0.5, proving the property